

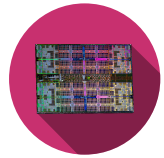
Architecture RISC-V

Labo Architecture des ordinateurs

Contenu

1	Introduction	2
2	Single-Cycle	3
2.1	Setup Ripes	3
2.2	Identification des blocs	3
2.3	Analyse des signaux	4
2.4	Comportement de l'architecture	5
3	Limitations de l'architecture SCR	7
4	Multi-cycle / Pipeline	8
4.1	Multi-cycle	8
4.2	Pipeline	9
4.3	Flush et hazards	9





1 Introduction

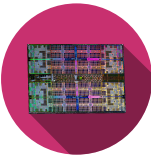
Ce laboratoire porte sur l'étude d'un cœur [Reduced Instruction Set Computer \(RISC-V\)](#) simple, d'abord dans une organisation single-cycle, puis dans des variantes multi-cycle et pipelinées. L'objectif est de comprendre comment les blocs de calcul, de mémoire et de contrôle coopèrent, ainsi que les conséquences de chaque choix architectural sur le temps d'exécution et le débit d'instructions.

À la fin du laboratoire, vous devez être capables de:

- identifier les blocs d'un processeur RISC-V simple et leur rôle
- relier les signaux de contrôle et de données à leur taille et à leur effet
- estimer le temps d'exécution d'une instruction **lw** et la fréquence maximale associée
- expliquer pourquoi un passage vers une architecture multi-cycle, puis pipelinée, est pertinent

Les activités sont organisées en trois parties:

- [Chapitre 2](#): découverte d'une architecture single-cycle simplifiée
- [Chapitre 3](#): estimation temporelle de l'instruction **lw** et discussion architecturale
- [Chapitre 4](#): étude d'une architecture pipelinée à 5 étages sans forwarding ni hazard detection.



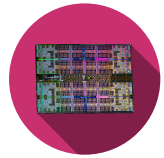
2.3 Analyse des signaux



Complétez le tableau suivant à partir de Ripes

Signal	Source	Taille en bits	Fonction
PC	Program Counter	32	Pointe la mémoire vers l'instruction courante
Instr.	Instruction Memory		
Reg1	Registers		
Reg2	Registers		
Immediate	Imm.		
Result	ALU		

Table 1 - Analyse guidée des signaux



2.4 Comportement de l'architecture

Exécutez quelques instructions représentant différents formats à l'aide du code suivant:

```
# Adapted from Ripes example:
# https://github.com/mortbopet/ripes/blob/master/examples/assembly/factorial.s
# Goal: show jal, stack use, branches, and register/dataflow updates.

.data
argument: .word 5

.text
main:
    lw a0, argument
    jal ra, fact
    # Display result
    li a7, 1
    ecall
    # Exit program
    li a7, 10
    ecall

fact:
    li t0, -16
    add sp, sp, t0
    sw ra, 8(sp)
    sw a0, 0(sp)
    addi t0, a0, -1
    bge t0, zero, nfact

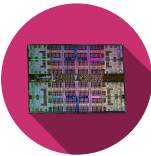
    addi a0, zero, 1
    addi sp, sp, 16
    jr x1

nfact:
    addi a0, a0, -1
    jal ra, fact
    addi t1, a0, 0
    lw a0, 0(sp)
    lw ra, 8(sp)
    addi sp, sp, 16
    mul a0, a0, t1
    ret
```



Relevez pour les instructions dans [Table 2](#) lors de leur première occurrence:

1. la valeur du PC avant/après exécution
2. les entrées/sorties de l'ALU
3. les autres signaux et multiplexeurs actifs liés au type d'instruction



Instruction	Ins- truktion	PC avant	PC après	ALU in1	ALU in2	ALU out	Signaux de contrôle actifs
<code>add sp, sp, t0</code>		0x00000020	0x00000024	0x7FFFFFF0 = Reg1	0xFFFFFFFF0 = Reg1	0x7FFFFFFE0 = ADD	W_{EnRegs} $Result_{Src} = ALU$ $PC_{Next} = PC + 4$
<code>sw ra, 8(sp)</code>							
<code>addi t0, a0, -1</code>							
<code>bge t0, zero, nfact</code>							
<code>jr x1</code>							
<code>lw a0, 0(sp)</code>							
<code>mul a0, a0, t1</code>							

Table 2 - Analyse des instructions exécutées

Limitations de l'architecture SCR

Cette partie utilise l'architecture de référence vue en cours pour estimer le temps de propagation critique d'une instruction **lw** en single-cycle.

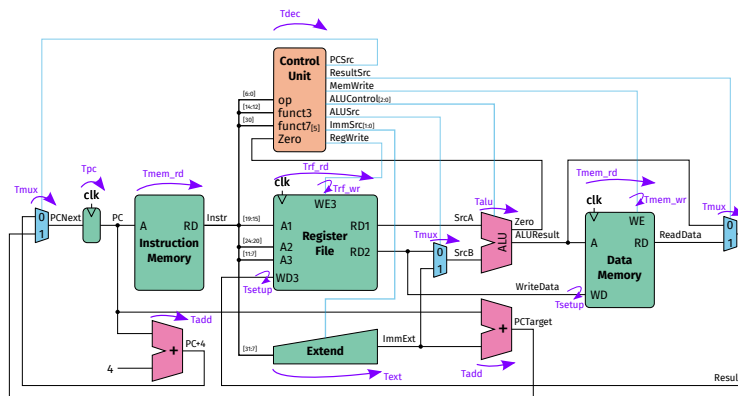


Figure 4 - Datapath de référence pour l'étude temporelle

Bloc	Délai	Description
tMux	30 ps	Routage/multi-plexage
tPC	40 ps	Mise à jour du PC
tAdd	100 ps	Addition 32 bits
tMemRd	200 ps	Lecture mémoire
tDec	25 ps	Décodage
tRfRd	100 ps	Lecture registre
tRfWr	60 ps	Écriture registre
tSetup	50 ps	Setup d'un registre
tExt	35 ps	Extension immédiate
tALU	120 ps	Opération ALU
tMemWr	60 ps	Écriture mémoire

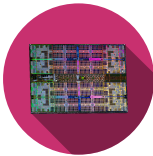
Table 3 - Délais imposés par bloc

Les flèches violettes traversant les blocs comme T_{MemRd} représentent le temps nécessaire à la sortie pour être valide après qu'une entrée ait chang  e.

Les flèches sur les signaux comme T_{setup} représentent le temps nécessaire pour que la valeur d'entrée puisse être enregistrée par un flanc d'horloge.

- 

$$f_{\max} = \frac{1}{T_{\text{cycle}}}.$$



4 | Multi-cycle / Pipeline

Dans cette partie, configurez Ripes avec un processeur « 5-stage processor w/o forwarding or hazard detection » - [Figure 2](#).

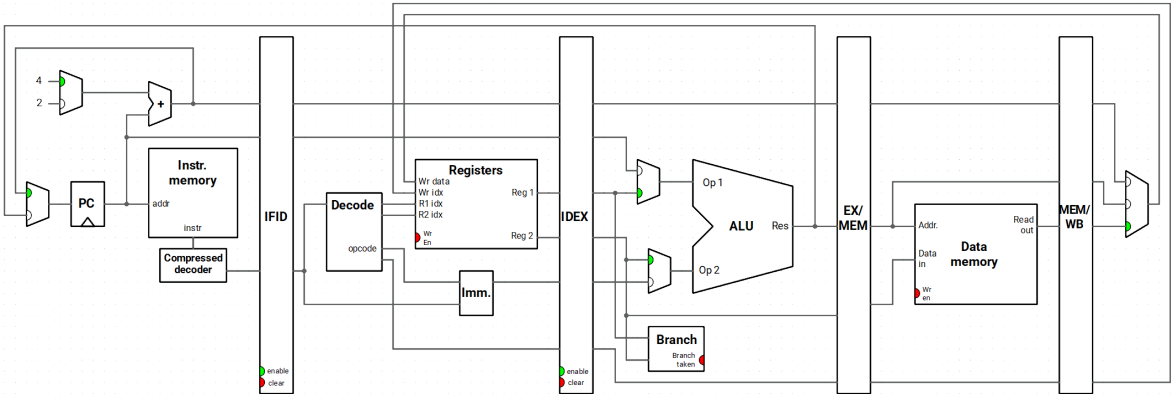


Figure 5 - Vue du pipeline 5 étaps dans Ripes

4.1 Multi-cycle

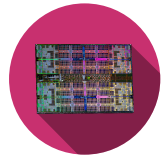
Formalisez le fonctionnement d’une architecture **multi-cycle**.



1. En vous appuyant sur le gabarit [Table 4](#), dessinez un chronogramme pour un code contenant, en se suivant, **add**, **lw** puis **sw**. Indiquez pour chaque coup d’horloge quelle(s) bloc(s) exécute(nt) quelle(s) instruction(s).
2. Pour chaque instruction, indiquez la latence (nombre de cycles pour une instruction).
3. Indiquez le débit moyen de ces trois instructions.
4. Quels avantages apporte l’architecture multi-cycles ?

Etapes	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15
Fetch	add														
Decode															
Execute															
Memory															
Write-back															

Table 4 - Diagramme multi-cycle



4.2 Pipeline

L'architecture pipeline s'inspire du fonctionnement multi-cycle en améliorant le débit en exécutant plusieurs instructions en parallèle.



1. En vous appuyant sur le gabarit [Table 5](#), dessinez un chronogramme pour un code contenant, en se suivant, **add**, **lw** puis **sw**. Indiquez pour chaque coup d'horloge quelle(s) bloc(s) exécute(nt) quelle(s) instruction(s).
2. Pour chaque instruction, indiquez la latence (nombre de cycles pour une instruction).
3. Indiquez le débit moyen de ces trois instructions.
4. Quel avantage introduit le pipeline par rapport à l'architecture multi-cycle ? Quels inconvénients apporte-t-il ?

Etapes	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15
Fetch	add														
Decode															
Execute															
Memory															
Write-back															

Table 5 - Diagramme pipeline

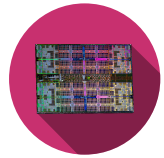
4.3 Flush et hazards

Le pipeline introduit des problèmes (**hazards**) lorsque des instructions dépendent de résultats d'instructions précédentes. Dans ce cas, le pipeline doit être « flushé » pour éviter des erreurs de calcul.

C'est notamment le cas pour les instructions de saut où les instructions précédemment chargées doivent être annulées si le saut est pris.

Exécutez le code suivant dans Ripes en observant l'état du pipeline à chaque étape:

```
# Ripes pipeline demonstration: branch / control-hazard penalty
#
# Purpose:
#   Show what happens to a pipeline when a conditional branch changes the
#   program counter.
#
# The loop executes three times:
#
#   s0 = 3 -> decrement to 2 -> branch TAKEN
#   s0 = 2 -> decrement to 1 -> branch TAKEN
#   s0 = 1 -> decrement to 0 -> branch NOT TAKEN
#
# In a pipelined processor, instructions following a branch may already have
# been fetched before the branch outcome is known. When the branch is taken,
# those incorrectly fetched instructions may have to be flushed.
```



```
#
# The NOPs after the loop are intentional. They make instructions following
# the branch visible in Ripes and help illustrate where execution continues.
#
# NOP is an alias for:
#   addi x0, x0, 0
#
# There is no meaningful program output; the interesting part is the
# pipeline timing and possible branch-flush penalty.

.text

main:
    li    t0, 1          # first input value
    li    t1, 2          # second input value
    li    s0, 3          # loop counter: execute the loop 3 times

loop:
    # Five independent additions.
    # These instructions do not depend on one another, so a pipeline can
    # overlap their execution.
    add    t2, t0, t1     # t2 = 1 + 2 = 3
    add    t3, t0, t1     # t3 = 1 + 2 = 3
    add    t4, t0, t1     # t4 = 1 + 2 = 3
    add    t5, t0, t1     # t5 = 1 + 2 = 3
    add    t6, t0, t1     # t6 = 1 + 2 = 3

    addi   s0, s0, -1     # decrement loop counter
    bnez   s0, loop       # if s0 != 0, jump back to loop

    # Deliberate NOPs: no useful work is performed here.
    # They make the instructions after the branch easy to see in Ripes.
    nop
    nop
    nop
    nop

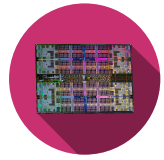
    ecall                # terminate program
```

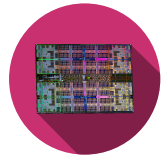


1. Itérez le code pas à pas. A chaque arrivée sur l'instruction de saut **bnez s0, loop**, observez le comportement du pipeline. Que se passe-t'il ?
2. En arrivant pour la dernière fois sur l'instruction de saut, observez le comportement du pipeline. Que se passe-t'il de différent ?
3. Expliquez pourquoi le pipeline n'est pas « 5 fois plus rapide » en pratique.



Ce chapitre introduit des concepts avancés servant aujourd'hui à optimiser les architectures processeurs. L'objectif est d'introduire l'idée de pipeline et ses limites pratiques, sans analyser tous les détails micro-architecturaux.





A Full-size LW datapath

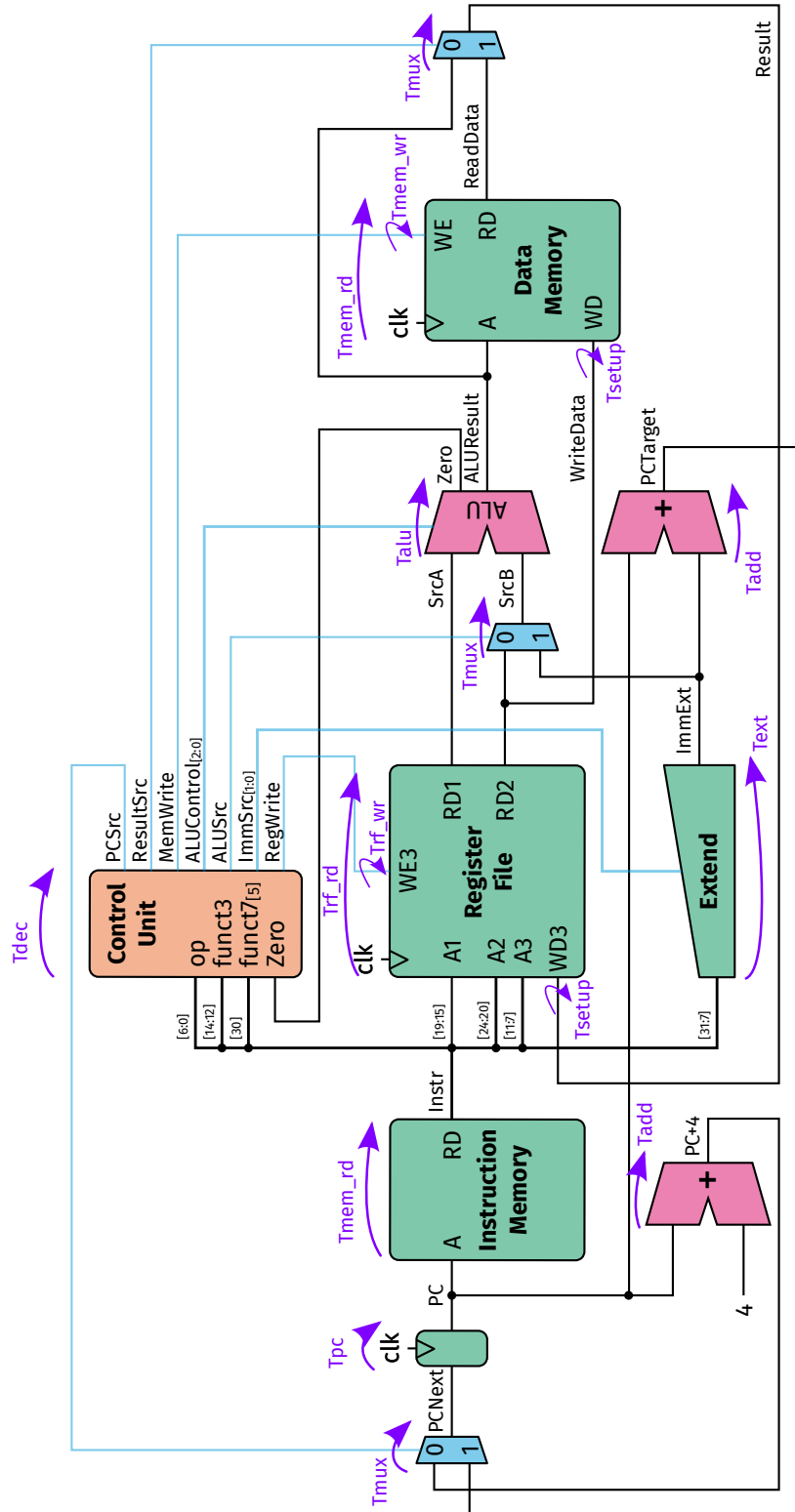


Figure 6 - Full-size reference datapath for LW timing